

SKILL WEEK11

TASK1: To connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
rishill10@ASUS16X: ~/Skill11
GNU nano 8.7.1 calculator.h *
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);

#endif
```

```
rishill10@ASUS16X: ~/Skill11
GNU nano 8.7.1 calculator.c *
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}
```

```
rishill10@ASUS16X: ~/Skill11
GNU nano 8.7.1 main.c *
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 10;
    int b = 5;

    printf("Addition = %d\n", add(a, b));
    printf("Subtraction = %d\n", subtract(a, b));

    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill11 x + v - □ ×  
rishill10@ASUS16X:~$ mkdir Skill11  
cd Skill11  
rishill10@ASUS16X:~/Skill11$ nano calculator.h  
rishill10@ASUS16X:~/Skill11$ nano calculator.c  
rishill10@ASUS16X:~/Skill11$ nano main.c  
rishill10@ASUS16X:~/Skill11$ gcc main.c calculator.c -o program  
./program  
Addition = 15  
Subtraction = 5  
rishill10@ASUS16X:~/Skill11$ |
```

TASK2: To detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
rishill10@ASUS16X: ~/Skill11 x + v - □ ×  
GNU nano 8.7.1 error_handling.c *  
#include <stdio.h>  
  
int main()  
{  
    int a, b;  
  
    printf("Enter two integers: ");  
  
    if (scanf("%d %d", &a, &b) != 2)  
    {  
        printf("ERROR: Invalid input. Please enter integers.\n");  
        return 1;  
    }  
  
    if (b == 0)  
    {  
        printf("ERROR: Division by zero is not allowed.\n");  
        return 1;  
    }  
  
    printf("Result = %d\n", a / b);  
  
    return 0;  
}|
```

```
rishill10@ASUS16X: ~/Skill11 x + v - □ ×  
rishill10@ASUS16X:~/Skill11$ nano error_handling.c  
rishill10@ASUS16X:~/Skill11$ gcc error_handling.c -o error_handling  
rishill10@ASUS16X:~/Skill11$ ./error_handling  
Enter two integers: 10 2  
Result = 5  
rishill10@ASUS16X:~/Skill11$ ./error_handling  
Enter two integers: 10 0  
ERROR: Division by zero is not allowed.  
rishill10@ASUS16X:~/Skill11$ ./error_handling  
Enter two integers: abc xyz  
ERROR: Invalid input. Please enter integers.  
rishill10@ASUS16X:~/Skill11$ ./error_handling > output.log 2>&1  
cat output.log  
rishill10@ASUS16X:~/Skill11$ |
```

TASK3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
rishill10@ASUS16X: ~/Skill11 x + v - □ X
GNU nano 8.7.1 leak.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr = (int *)malloc(sizeof(int) * 10);
    *ptr = 42;
    printf("Value: %d\n", *ptr);
    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill11 x + v - □ X
GNU nano 8.7.1 leak.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr = (int *)malloc(sizeof(int) * 10);
    *ptr = 42;
    printf("Value: %d\n", *ptr);
    free(ptr);
    return 0;
}
```

```
rishill10@ASUS16X: ~/Skill11
rishill10@ASUS16X:~/Skill11$ nano leak.c
rishill10@ASUS16X:~/Skill11$ gcc -g leak.c -o leak
rishill10@ASUS16X:~/Skill11$ valgrind --leak-check=full ./leak
==740== Memcheck, a memory error detector
==740== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==740== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==740== Command: ./leak
==740==
Value: 42
==740==
==740== HEAP SUMMARY:
==740==   in use at exit: 40 bytes in 1 blocks
==740==   total heap usage: 2 allocs, 1 frees, 1,064 bytes allocated
==740==
==740== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==740==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==740==    by 0x400117E: main (leak.c:6)
==740==
==740== LEAK SUMMARY:
==740==    definitely lost: 40 bytes in 1 blocks
==740==    indirectly lost: 0 bytes in 0 blocks
==740==    possibly lost: 0 bytes in 0 blocks
==740==    still reachable: 0 bytes in 0 blocks
==740==    suppressed: 0 bytes in 0 blocks
==740==
==740== For lists of detected and suppressed errors, rerun with: -s
==740== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
rishill10@ASUS16X:~/Skill11$ nano leak.c
rishill10@ASUS16X:~/Skill11$ gcc -g leak.c -o leak
rishill10@ASUS16X:~/Skill11$ valgrind --leak-check=full ./leak
==774== Memcheck, a memory error detector
==774== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==774== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==774== Command: ./leak
==774==
Value: 42
==774==
==774== HEAP SUMMARY:
==774==   in use at exit: 0 bytes in 0 blocks
==774==   total heap usage: 2 allocs, 2 frees, 1,064 bytes allocated
==774==
==774== All heap blocks were freed -- no leaks are possible
==774==
==774== For lists of detected and suppressed errors, rerun with: -s
==774== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
rishill10@ASUS16X:~/Skill11$ |
```

TASK4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
rishill10@ASUS16X: ~/Skill11 × + v
GNU nano 8.7.1 test_script.sh *
#!/bin/bash

echo "Test Report" > test_report.txt
echo "===== " >> test_report.txt
echo "Starting automated tests for error_handling..." >> test_report.txt

test_case() {
    local input_data="$1"
    local expected_output="$2"
    local test_name="$3"

    actual_output=$(echo "$input_data" | ./error_handling)

    if [ "$actual_output" == "$expected_output" ]; then
        echo "PASS: $test_name" >> test_report.txt
    else
        echo "FAIL: $test_name" >> test_report.txt
        echo "    Expected: $expected_output" >> test_report.txt
        echo "    Actual:   $actual_output" >> test_report.txt
    fi
}

test_case "10 2" "Enter two integers: Result = 5" "Valid Input (10 / 2)"
test_case "10 0" "Enter two integers: ERROR: Division by zero is not allowe>
test_case "abc xyz" "Enter two integers: ERROR: Invalid input. Please enter>

echo "===== " >> test_report.txt
echo "Testing complete." >> test_report.txt

cat test_report.txt|
```

```
rishill10@ASUS16X: ~/Skill11 × + v
rishill10@ASUS16X:~/Skill11$ gcc error_handling.c -o error_handling
rishill10@ASUS16X:~/Skill11$ nano test_script.sh
rishill10@ASUS16X:~/Skill11$ chmod +x test_script.sh
rishill10@ASUS16X:~/Skill11$ ./test_script.sh
Test Report
=====
Starting automated tests for error_handling...
PASS: Valid Input (10 / 2)
PASS: Division by Zero
PASS: Invalid Syntax
=====
Testing complete.
rishill10@ASUS16X:~/Skill11$ |
```